

Cartilha para o Teste Técnico

Parabéns por chegar até aqui! Para ajudá-lo a se preparar melhor para o teste técnico, elaboramos este guia com as principais boas práticas e critérios que serão avaliados no seu código. Siga essas diretrizes para garantir uma boa performance e aumentar suas chances de sucesso.

1. Funcionalidades Essenciais

CRUD Completo: Sua aplicação deve permitir criar, ler, atualizar e excluir registros para as entidades principais.

Modelo de Dados Coerente: Utilize uma estrutura de banco de dados bem definida, respeitando boas práticas de modelagem.

2. Regras de Negócio

Desconto de 10%: Aplique 10% de desconto quando um determinado critério for atendido.

Desconto de 20%: Aplique 20% de desconto quando outro critério específico for atendido.

Limite de 20 Itens Iguais: Um pedido não pode conter mais de 20 unidades do mesmo item.

Mínimo de Itens para Desconto: Descontos só devem ser aplicados caso um número mínimo de itens seja adquirido.

3. Arquitetura e Organização

Divisão em Camadas: Utilize uma estrutura organizada separando controllers, services e repositórios.

Publicação de Eventos: Se possível, implemente um sistema de eventos para comunicação entre diferentes partes da aplicação.

4. Boas Práticas de Código

Clean Code: Utilize nomes descritivos, evite código duplicado e siga boas práticas de organização.

SOLID: Aplique os princípios SOLID para manter um código modular e sustentável.

DRY (Don't Repeat Yourself): Evite repetição desnecessária de código.

YAGNI (You Ain't Gonna Need It): Não implemente funcionalidades que não foram requisitadas.

Object Calisthenics: Pratique técnicas para tornar o código mais legível e flexível.

5. Logs e Monitoramento

Serilog: Implemente logs estruturados usando Serilog para facilitar depuração e monitoramento.

6. Boas Práticas de Git

Git Flow: Siga uma estratégia organizada para gerenciar suas branches.

Commit Semântico: Utilize mensagens de commit claras e padronizadas.

7. Padrões RESTful

Certifique-se de que sua API segue os princípios RESTful, utilizando corretamente os métodos HTTP (GET, POST, PUT, DELETE) e retornando os códigos de status apropriados.

8. Testes Automatizados

XUnit: Escreva testes unitários para validar a funcionalidade do seu código.

Fluent Assertions: Utilize esta biblioteca para melhorar a legibilidade dos testes.

Bogus: Gere dados fictícios para testes automatizados.

NSubstitute: Use mocks para testar componentes isoladamente.

Testing Containers: Utilize containers para testes de integração.

9. Melhores Práticas Gerais

Escreva código pensando na manutenção e escalabilidade.

Priorize a legibilidade e clareza em sua implementação.

Teste sua aplicação antes de enviar o código.

Seguindo essas diretrizes, você terá um código bem estruturado e que atende aos requisitos da avaliação. Boa sorte e sucesso no teste!